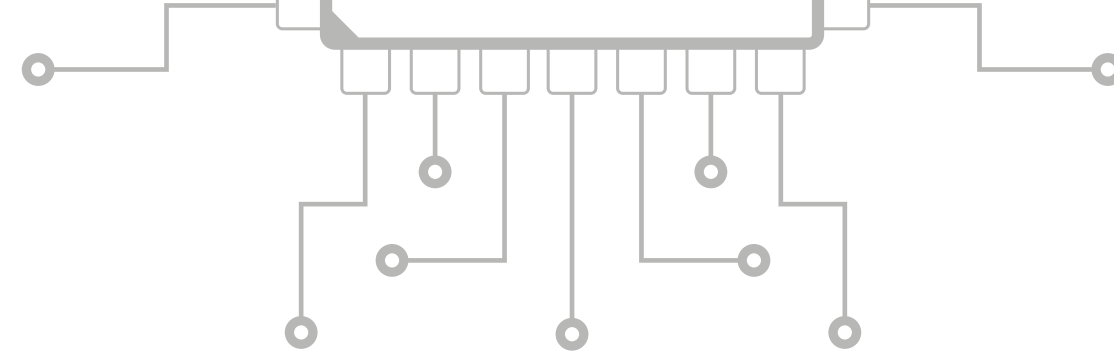
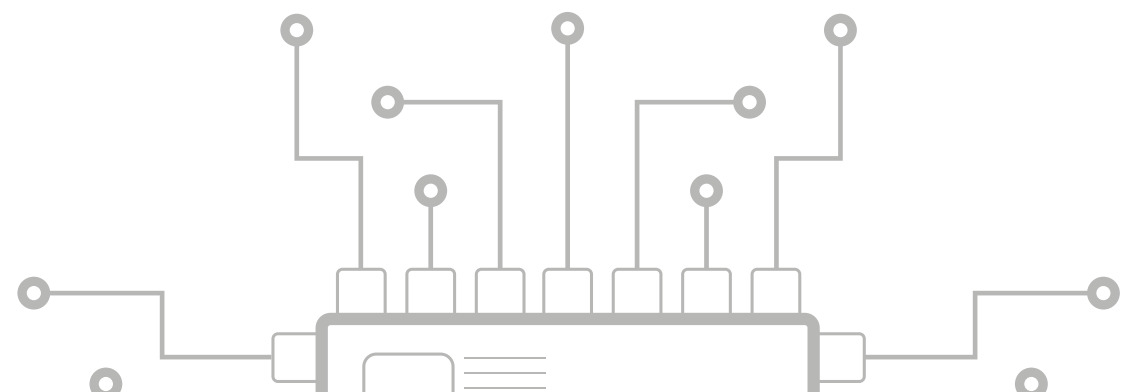




Úvod do programování v jazyce C

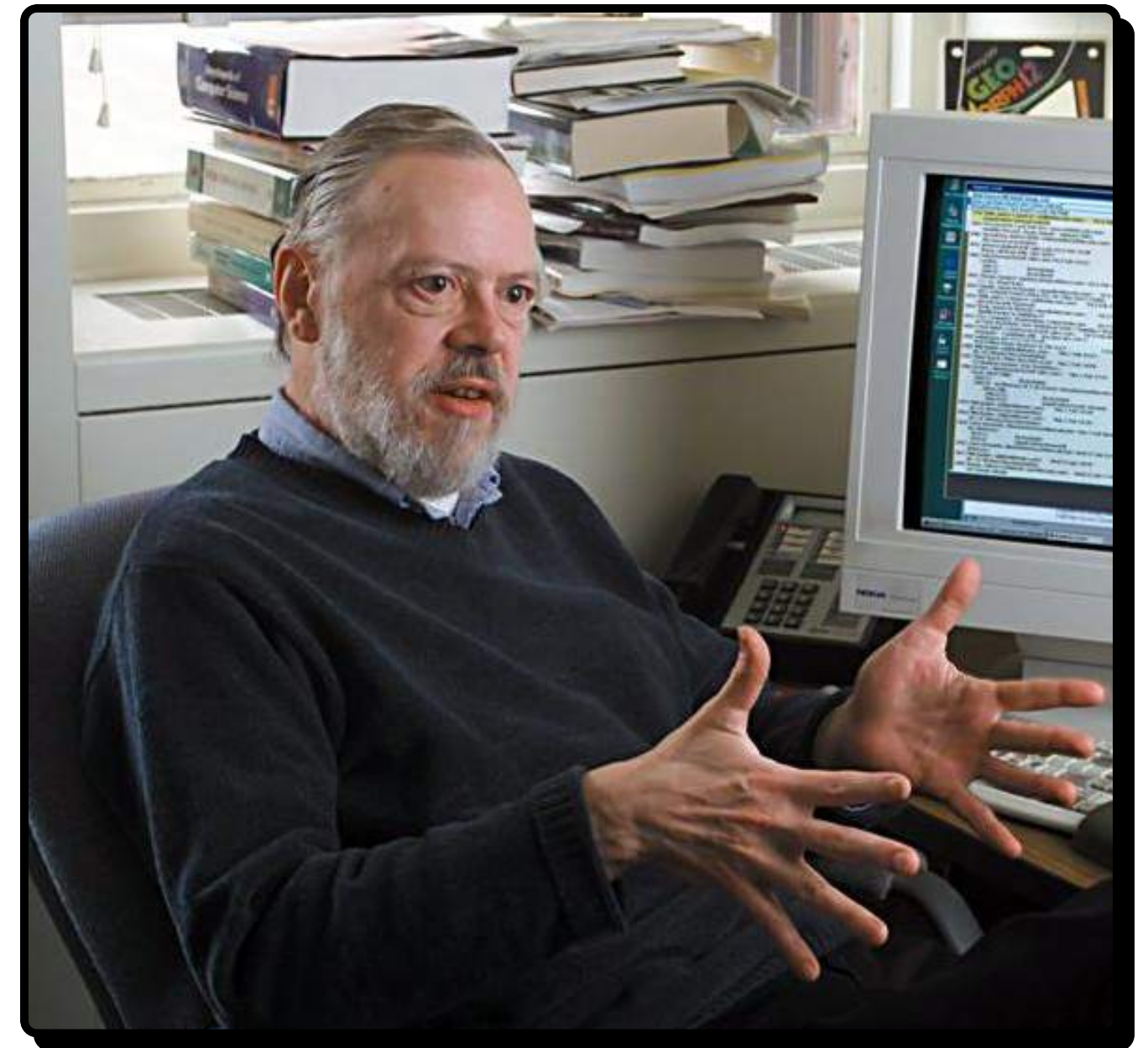
Specifika jazyka, výhody/nevýhody,
využití, IDE, struktura kódu

Dominik Němeček



Jazyk C

- Jazyk C byl **vyvinut Dennisem Ritchiem v roce 1972** v Bell Labs jako vylepšení jazyka B.
- Byl navržen pro vývoj operačního systému Unix, který díky tomu získal popularitu, a **C se rozšířilo jako standardní jazyk pro systémové programování.**
- **Využití:** Vývoj operačních systémů (Linux, Windows), programování embedded systémů a mikrokontrolérů, tvorba síťových aplikací, herních engineů a databázových systémů, knihovny a systémový software



Dennis Ritchie

Verze jazyka C

- **K&R C (1978)**: Původní verze popsaná Kernighanem a Ritchiem.
- **ANSI C (C89)**: První standard z roku 1989.
- **C99**: Verze z roku 1999, přidávající nové datové typy a vylepšení.
- **C11**: Standard z roku 2011 s podporou multi-threadingu a Unicode.
- **C18**: Aktualizace z roku 2018 zaměřená na opravy a stabilitu.



Specifika jazyka C

1

Nízká abstrakce

Umožňuje práci s pamětí a hardwarem.

2

Strukturované prg.

Podporuje funkce, cykly a podmínky.

3

Přenositelnost

Kód lze snadno přenést mezi platformami.

4

Statické typování

Proměnné musí mít definovaný typ.

5

Ukazatelé

Umožňuje samostatnou správu paměti.

6

Minimální runtime

C je rychlý a nenáročný na systémové zdroje.

7

Knihovny

Poskytuje knihovny pro běžné operace.

8

Kompatibilita

Kompatibilita se starším kódem.

Výhody a nevýhody C



Vysoký výkon

Výkonný jazyky díky kompilaci.



Přístup k HW

Přímý přístup k HW pomocí ukazatelů.



Přenositelnost

Kód lze snadno přenést mezi platformami.



Podpora

Dlouhodobě populární jazyk.



Nepodporuje OOP

Nepodporuje koncept programování OOP.



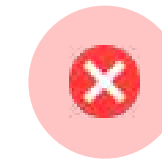
Chybí GC

Bez automatické správy paměti.



Složitost

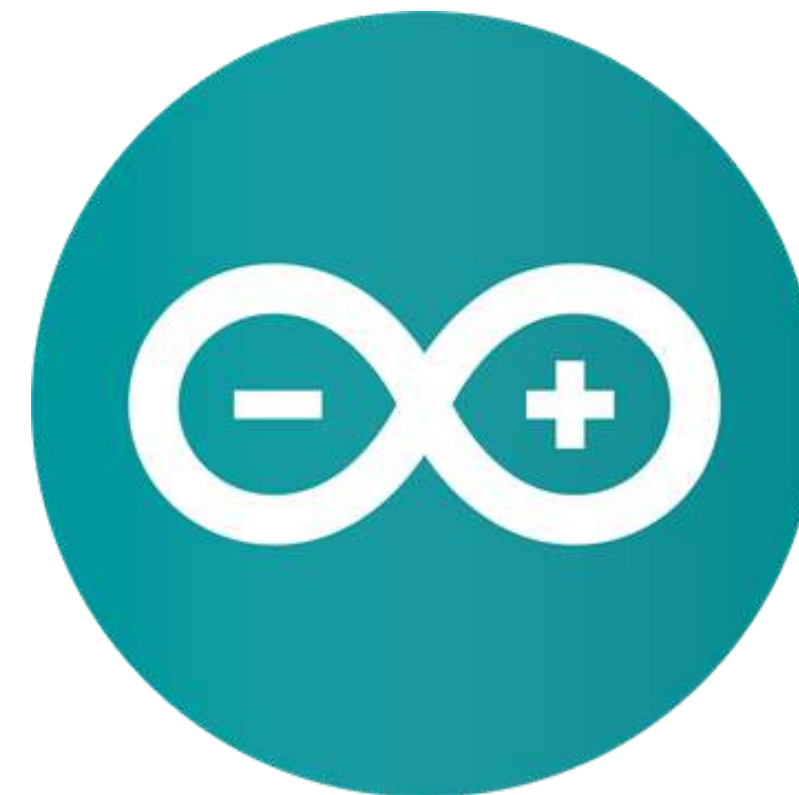
Složitá práce z důvodu ukazatelů.



Nízká abstrakce

Menší komfort pro programátora.

Proč se učíme jazyk C?



Vývojové prostředí (IDE)

IDE:

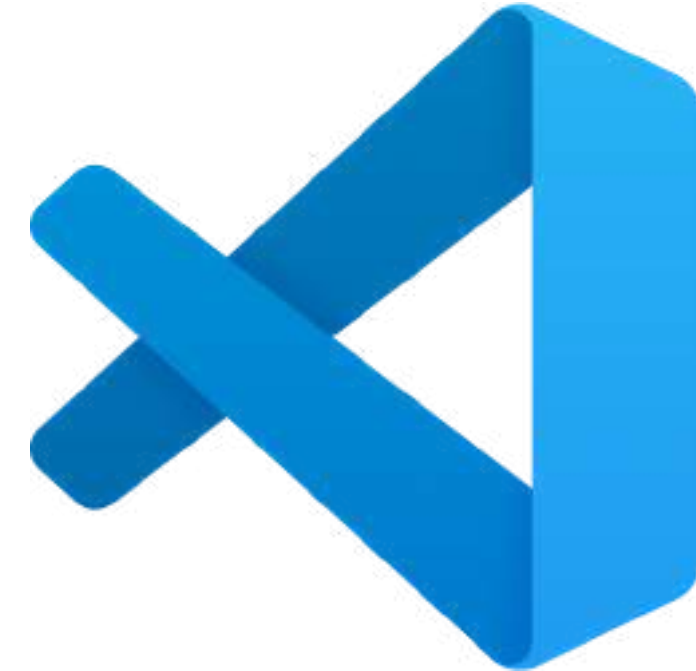
- Vývojové prostředí (Integrated Development Environment, IDE) je software, který usnadňuje programátorům tvorbu aplikací.

IDE obvykle zahrnuje několik klíčových nástrojů v jednom balíčku:

- Editor kódu, překladač, ladící nástroje, integrovaná dokumentace

Druhy IDE pro C:

- Visual Studio Code, CLion, Netbeans



Vývojové prostředí pro C

NetBeans (Apache):

- **Popis:** NetBeans je open-source vývojové prostředí, které podporuje různé jazyky, včetně Javy, PHP, a C/C++. Je vhodné pro vývoj aplikací, webů a mobilních řešení díky široké škále vestavěných nástrojů a modulů.
- **Funkce:** Podpora pro různé jazyky, integrované nástroje pro debugging, testování, správu projektů, podpora verzovacích systémů (Git, Mercurial) a podpora vývoje GUI aplikací.
- **Platforma:** Windows, macOS, Linux.

[!\[\]\(3d8c13c92b853674f749aac6fa869926_img.jpg\) Odkaz ke stáhnutí IDE](#)



Vývojové prostředí pro C

Visual Studio Code (Microsoft):

- **Popis:** Visual Studio Code je lehký, multiplatformní editor, který je možné přizpůsobit pomocí rozšíření. Je ideální pro vývoj v C s použitím různých rozšíření.
- **Funkce:** Snadné použití, podpora pro rozšíření, integrovaný terminál, debugging, podpora pro Git a další nástroje.
- **Platforma:** Windows, macOS, Linux.

[!\[\]\(5ebcf382a6ee952d6c5b8b948415801e_img.jpg\) Odkaz ke stáhnutí IDE](#)



Vývojové prostředí pro C

CLion (JetBrains)

- **Popis:** CLion je vývojové prostředí zaměřené na programování v C a C++, které nabízí chytrou nápovědu při psaní kódu, pokročilé refaktorování a integraci různých nástrojů.
- **Funkce:** Pokročilá podpora pro C a C++, debugging, integrované nástroje pro testování, podpora verzovacích systémů jako Git, optimalizace kódu a analýza výkonu.
- **Platforma:** Windows, macOS, Linux.

[Odkaz ke stáhnutí IDE](#)



Základní rozbor kódu

Hlavní funkce `main()`

- Program se skládá z hlavní funkce, většinou `main()`.
- Ve funkci `main()` probíhá hlavní část programu.
- Datový typ typu `int`, konec programu `return 0` – úspěšné ukončení

```
#include <stdio.h>

int main()
{
    printf("Hello!");
    return 0;
}
```

Hlavní funkce programu

Základní rozbor kódu

Knihovny

- Knihovna je **předpřipravený soubor kódu obsahující funkce nebo prvky**, které lze opakovaně využívat v programech.
- Úspornost kódu, opakující se funkčnosti, modularita.
- **#include <stdio.h>** vs **#include "soubor.h"**

Základní knihovna



```
#include <stdio.h>

int main()
{
    printf("Hello!");
    return 0;
}
```

Základní rozbor kódu

Typy závorek

- Závorky jsou vždy párové!
- **Špičaté <>**: používá se pro knihovny
- **Složené {}**: používá se pro bloky kódu
- **Kulaté ()**: Používá se pro funkce
- **Hranaté []**: Používá se pro manipulaci s polem

Špičaté závorky

```
#include <stdio.h>
```

Kulaté závorky

```
int main()  
{  
    printf("Hello!");  
    return 0;  
}
```

Složené závorky

Základní rozbor kódu

Výpis textu na konzoli

- Pokud chceme vypsát text nebo obsah proměnné do konzole, použijeme k tomu funkci **printf()**.
- Text ve funkci se píše do dvojitého uvozovky: "Text".
- Pokud budeme chtít odřádkovat daný text, použijeme formátovací značku **\n**.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
printf("Hello!\n");
```

```
return 0;
```

```
}
```

Výpis textu s odřádkováním

Základní rozbor kódu

Komentáře

- Důležitá část kódu – pro lepší čitelnost a pochopitelnost.
- Existují **jednořádkové a víceřádkové komentáře**.

```
// Jednořádkový komentář
```

```
/*  
Víceřádkový komentář  
*/
```

Jednořádkový komentář

```
#include <stdio.h>  
// Hlavní funkce  
int main()  
{  
    printf("Hello! \n");  
    return 0;  
}
```

Základní rozbor kódu

Na co si dávat pozor?

- Nezapomenout psát středník na konci řádku (existují případy, kde se nepíše).
- Uzavírat všechny párové závorky.
- Psát dvojité uvozovky v textovém řetězci.
- Nezapomínat psát základní hlavičkový soubor pro používání knihovny.
- Nezapomínat postupně dokumentovat daný kód pomocí komentářů.